

The Non-Analytic Critical-Region Term

Part 3: Rust Implementation

outram-park-fork-coolprop

2026-07-10

Abstract

Parts 1–2 covered the theory and the numerical guard. This document covers how the non-analytic term is represented as data, dispatched, and evaluated in this crate’s Rust code — the enum-dispatch pattern this workspace uses everywhere in place of trait objects, and how a fluid’s hardcoded data feeds into it.

1 Where the data lives: an enum variant, not a class

Following this workspace’s mandatory “no trait objects” rule, every residual-Helmholtz term *type* CoolProp supports is a variant of a single enum, `ResidualTerm` (`src/eos.rs`):

```
pub enum ResidualTerm {
    Power { n: &'static [f64], d: &'static [f64], t: &'static [f64], l:
        &'static [f64] },
    Gaussian { n: &'static [f64], d: &'static [f64], t: &'static [f64],
        eta: &'static [f64], epsilon: &'static [f64],
        beta: &'static [f64], gamma: &'static [f64] },
    NonAnalytic { n: &'static [f64], a: &'static [f64], b: &'static [f64]
    },
    beta: &'static [f64], big_a: &'static [f64], big_b: &'
        static [f64],
    big_c: &'static [f64], big_d: &'static [f64] },
    // ... Exponential, DoubleExponential, GaoB
}
```

Each field is a `&'static [f64]` slice, not a `Vec` — the whole term list for a fluid is `const` data compiled directly into the binary (see Part 4’s `water.rs`), never read from a file at runtime. A fluid’s full equation of state, `FluidEos`, is simply `residual: &'static [ResidualTerm]` plus the ideal-gas terms and reducing/critical parameters.

2 Dispatch: an exhaustive match, not virtual calls

Evaluating a fluid’s residual Helmholtz derivatives at a state (δ, τ) walks the term list and dispatches on the enum:

```
pub fn accumulate(&self, delta: f64, tau: f64, acc: &mut HelmholtzDerivs) {
    match self {
        ResidualTerm::Power { n, d, t, l } => { /* ... */ }
        ResidualTerm::Gaussian { n, d, t, eta, epsilon, beta, gamma } =>
            { /* ... */ }
        ResidualTerm::NonAnalytic { n, a, b, beta, big_a, big_b, big_c,
            big_d } => {
```

```

        accumulate_non_analytic(n, a, b, beta, big_a, big_b, big_c,
                                big_d, delta, tau, acc);
    }
    // ...
}

```

This is the enum-dispatch pattern the whole workspace uses in place of `Box<dyn Trait>`: the compiler checks the `match` is exhaustive, so adding a new `CoolProp` term type without updating this function is a compile error rather than a silent runtime gap — and there is no heap allocation or virtual-call indirection, since every branch is known at compile time.

3 The non-analytic evaluator itself

`accumulate_non_analytic` takes the eight coefficient slices, the state (δ, τ) , and a mutable accumulator, applies the branch-point offset guard from Part 2, then loops over however many non-analytic terms the fluid has (Water has two) accumulating each term’s contribution:

```

for i in 0..n.len() {
    let (ni, ai, bi, betai) = (n[i], a[i], b[i], beta[i]);
    let (aai, bbi, cci, ddi) = (big_a[i], big_b[i], big_c[i], big_d[i]);
    let dm1 = delta - 1.0;
    let dm1_sq = dm1 * dm1;

    let theta = (1.0 - tau) + aai * dm1_sq.powf(1.0 / (2.0 * betai));
    // ... dtheta_ddelta, d2theta_ddelta2, psi and its derivatives,
    // big_delta and its derivatives, delta_bi = big_delta.powf(bi)
    // and ITS derivatives -- see Part 2's derivative chain --

    acc.add_scaled(&HelmholtzDerivs {
        a: delta * ni * delta_bi * psi,
        ad: /* ... */, at: /* ... */,
        add: /* ... */, att: /* ... */, addt: /* ... */,
    });
}

```

`acc.add_scaled` sums this term’s contribution into the running total for the whole residual sum — every term type ultimately contributes through the same six-field `HelmholtzDerivs` accumulator, which is what lets `state_rho` (Part 4) stay agnostic to which term types a particular fluid happens to use.

4 Why this design, not something more “generic”

A tempting alternative would be a trait `ResidualTermKind` with one implementor struct per term type, dispatched via `Box<dyn ResidualTermKind>`. This workspace deliberately avoids that (see the root `CLAUDE.md`’s “no trait objects” rule) for three concrete reasons that matter for a physics library specifically: (1) the set of term types is closed — `CoolProp`’s own generator (Part 4) already aborts loudly if a fluid ever needs an unimplemented type, so exhaustiveness-checking via `match` is strictly more valuable than open-ended extensibility; (2) every fluid’s entire EOS is `const` data with zero heap allocation, which a `Vec<Box<dyn Trait>>` would break; (3) go-to-definition on an enum variant works reliably in any IDE, while go-to-definition on a `dyn Trait` call site often does not — directly serving this workspace’s “human interface layer” design principle (also in the root `CLAUDE.md`) that any public API must be navigable by `rust-analyzer` alone.